Overview

🎯 In this course, you will learn how to interact with the AUT's objects using the Squish Object Map.

What Will I Learn ?

1. What is an object map in Squish, and what are its naming schemes?
2. How to create and edit object maps?
3. How do you interact with objects that are registered into the object map?
4. What is object name generation in Squish and how it works?
5. What are occurrences and how to avoid them?

[How to Identify and Access Objects | Squish 9.2]

[Object Map | Squish 9.2]

[Object Name Generation | Squish 9.2]

[GitHub - qt-learning/Object-Identification · GitHub]

The Concept of the Object Map

One of the most important concepts testers must consider when writing and modifying test scripts, is how to access objects in the user interface. Squish relies on the Object Map for that purpose (often called a GUI object map in QA literature).

The Object Map is a maintainable repository which defines names for the individual objects in an **Application Under Test (AUT)**.

Naming Schemes Used in the Object Map

Two naming schemes are used in the Object Map:

Symbolic Name	A string key which represents an AUT object. Example: **o2_Text**
Real Name (aka multi-property names)	A set of object properties used to identify an AUT object. Example: **{"container": o_QQuickView, "text": 2, "type": "Text", "visible": True}**

ℹ️ **Each entry in the Object Map consists of a pair symbolic name-real name.**

Test script code does not use the object names of the AUT directly. Instead, the Object Map associates each object name with a so-called symbolic name, which serves as a “key” into the Object Map. This “key” is mapped to an object in the application. This makes maintaining test scripts easier when the AUT, its object hierarchy, or its object names change.

ℹ️ **In most editions of Squish, symbolic names are used when recording scripts.**

Another benefit of the Object Map is that symbolic names are independent of the AUT. Thus, when the AUT changes, test scripts which use those symbolic names do not need any modification. Only the real names in the Object Map need to be modified if the AUTs changes.

Implementation of the Object Map Concept

There are multiple ways to implement the Object Map concept. The default method in Squish is the Script-Based Object Map.

Each Squish Test Suite manages an Object Map which contains the mapping for the Application Object Tree. Each entry in the map is referenced by a symbolic name, which stores a list of properties and their associated values, i.e., the real name.

When looking up an object, Squish will:

1. **Find symbolic name in the test script**

   When clicking a button during a test, the symbolic name “newButton”, corresponding to the button, is used in the test script as shown.

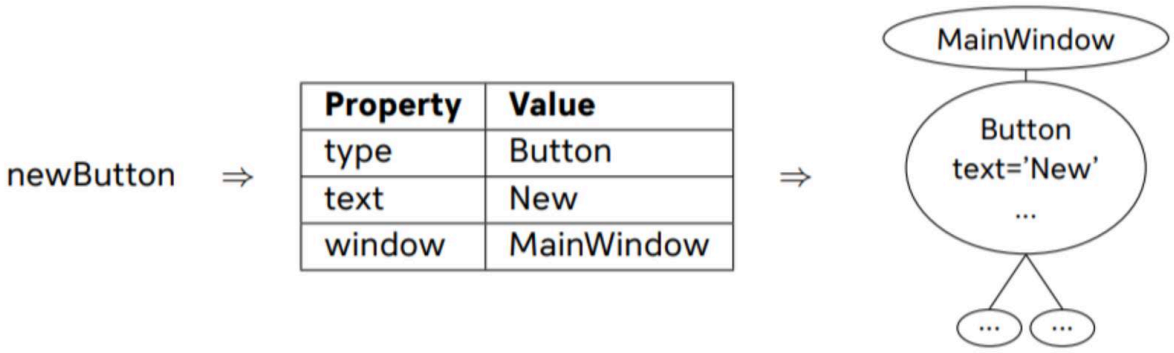
```
mouseClick(waitForObject(names.newButton))
```

2. **Lookup for symbolic properties and values in the Object Map corresponding to the symbolic name**

   Squish finds the symbolic properties associated with the symbolic name “newButton” as shown.

```
newButton = {"window": MainWindow, "text": New, "type": "Button"}
```

3. **Search in the AUT for an object that matches the properties and values**



tst_case (suite_myTestSuite)

```
1  # -*- coding: utf-8 -*-
2
3  import names
4
5
6  def main():
7      startApplication("addressbook")
8      clickButton(waitForObject(names.address_Book_New_QToolButton))
```

Find a symbolic name in the test script
(address_Book_New_QToolButton)

Once these steps are completed Squish has successfully identified an AUT object, based on the symbolic name. The button is clicked.

Creation and Edition of the Object Map

When creating a new Test Suite in Squish, a script called names (.py,.js,..based on the scripting language used in the Test Suite) is automatically generated in the shared/scripts folder. This is the default Script Object Map. Of course, it is possible to split this script into several other files for improving maintainability. It is possible to have several script files in shared/scripts that define the Object Map or even use scripts from a global script folder if these scripts are shared among several Object Maps.

The **names** file is initially empty. It is populated with pairs of symbolic and real names when the tester interacts with the AUT during recording, Spy the **AUT**, and/or adds objects via the Picker Tool. It is also possible to edit the Object Map manually via the Object Map view integrated in the Squish IDE or directly modify the script(s).

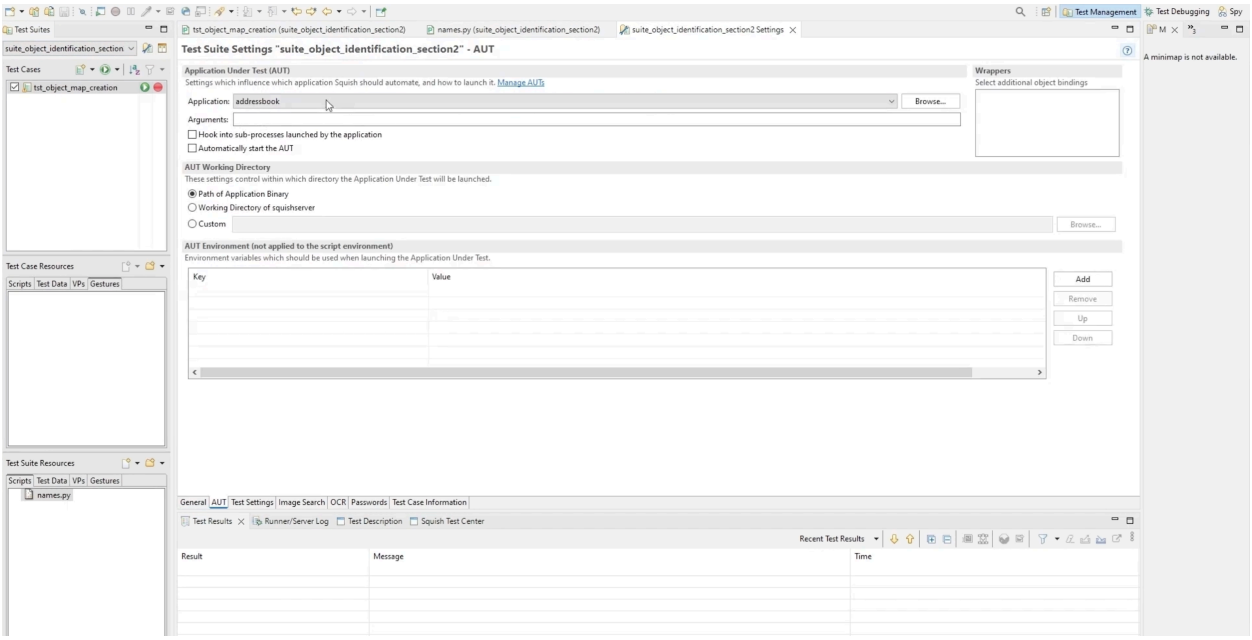
When starting from an empty test suite, the easiest and quickest approach to generate an Object Map is by recording a test. Squish will populate the Object Map during the recording. It is recommended to interact with every object planned to be used in the tests.

Once the Object Map has been generated, it can be managed in the Object Map view. All symbolic names and their properties can be viewed here. Name entries can be modified, added, or removed. All the modifications made from this view will be reflected in the names script.

This view can also be used to check the existence of an object, highlight it, and show it in the application context. When editing the Object Map from this view, it is also possible to let Squish automatically update the test scripts.

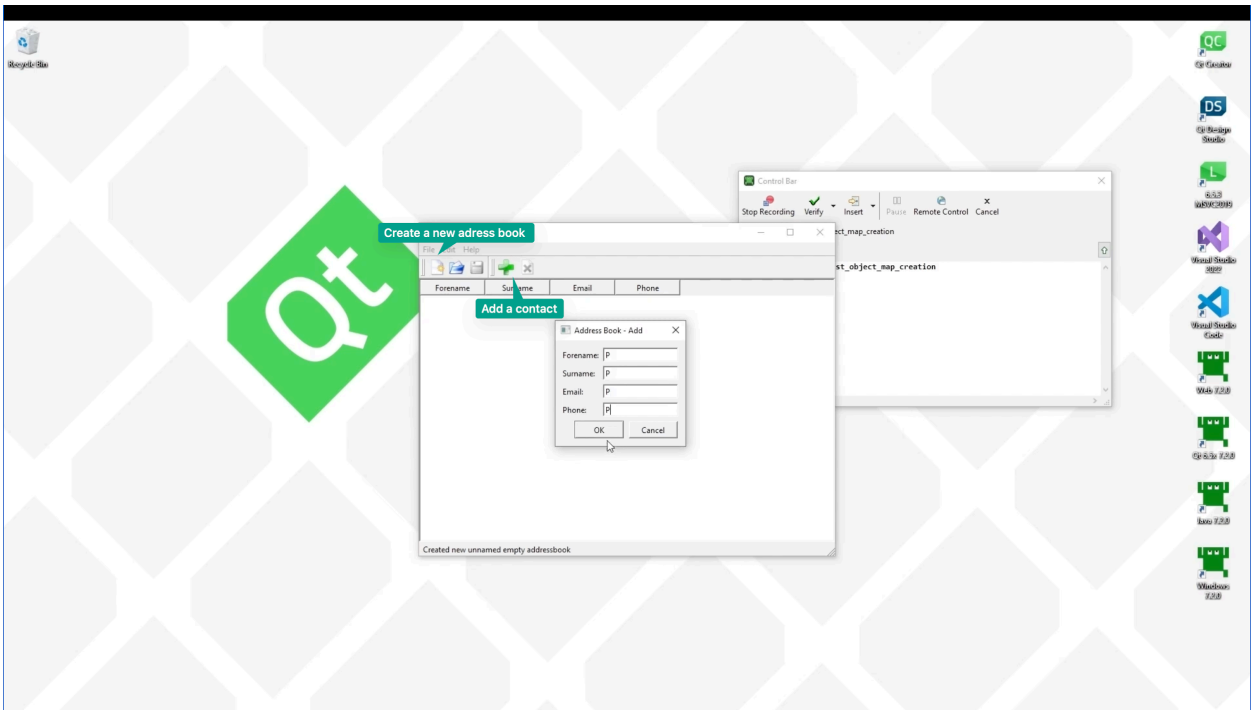
STEPS - How the Creation And Edition of the Object Map Works in the Squish IDE

Step 00 = Starting point



We used the Address Book application available in the examples folder of the Squish directory. We had created an empty test suite and could see the **names.py** file was generated and added to our Test Suite Resources. This was our script-based object map.

01 Recording a dummy test to generate the object map



We started by recording a dummy test case:

1. We clicked the record button.

2. Then, we started the AUT.

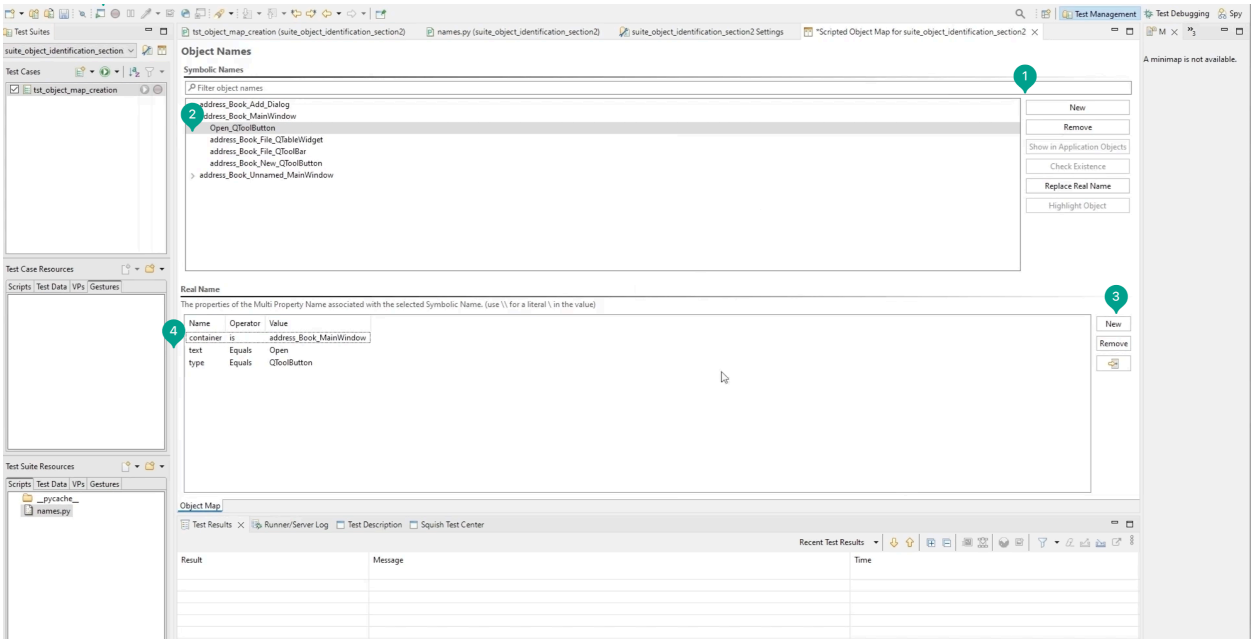
3. Finally, we created a new address book, and added a contact.

Then, we checked that the table was what you were expecting.

From the Control Bar Window, we selected Verify and Table. The Squish IDE opened. We selected QTableWidget from Address Book. Then, we selected Save and Insert Verification.

We stopped the recording, and we could see that the objects of the AUT were referred to in the test script with “names. Name of the object” (with their symbolic name). If we checked the Object Map view or names.py, we saw that those corresponded to real names, the set of properties and values Squish used to locate an object in the UI.

02 Adding a new entry to the object map in the object map view



1. We clicked on New to add an entry for the Open button.

2. Then, we typed Open_QToolButton for the name.

3. We selected New in the Real Name section three times.

4. We identified the created button with its type, text, and container window:

text equals Open

type equals QToolButton

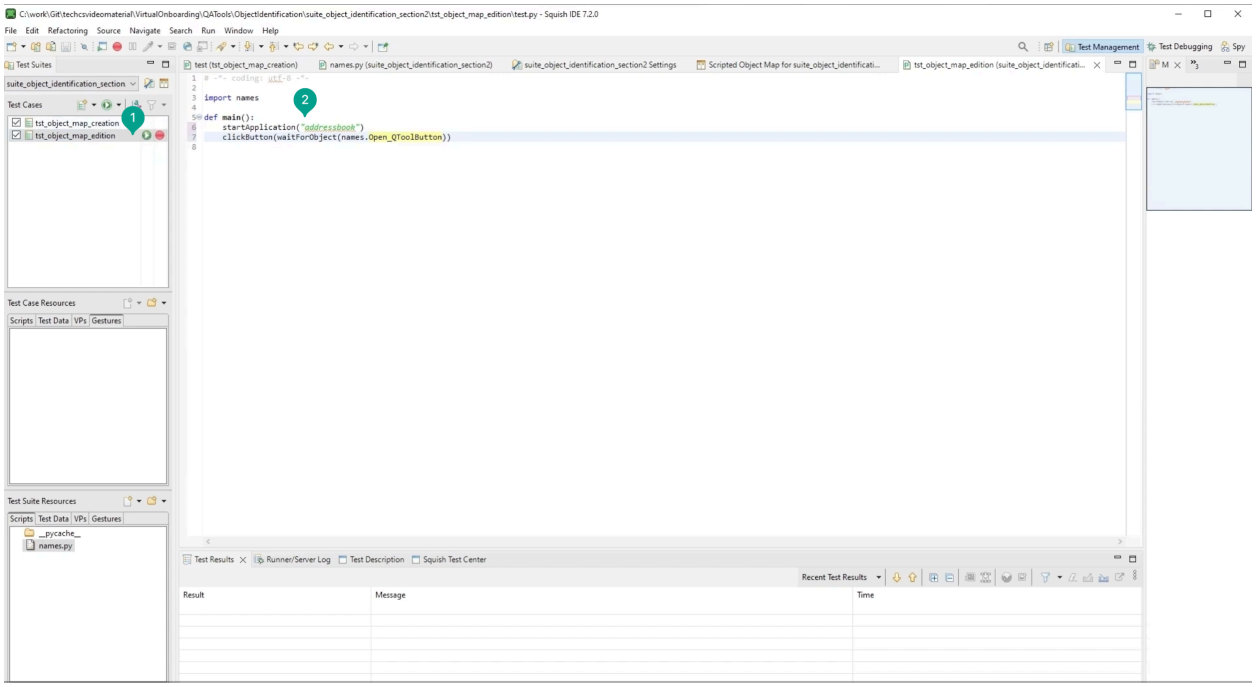
container is address_Book_MainWindow

5. Finally, we started the AUT and verified the Open button was correctly referenced.

It would have been also possible to show the object in the Application Objects tree, check its existence, or highlight the object from the Object map window. So, we started the AUT and verified the Open button was correctly referenced.

Note that you can also modify the Object Map from the script names.py. However, doing that will not update the test scripts automatically. In general, it is preferable to use the integrated Object Map view.

03 Writing a new test where we interacted with the open button



1. We wrote a new test “tst_object_map_edition”. In this, we interacted with the Open button.

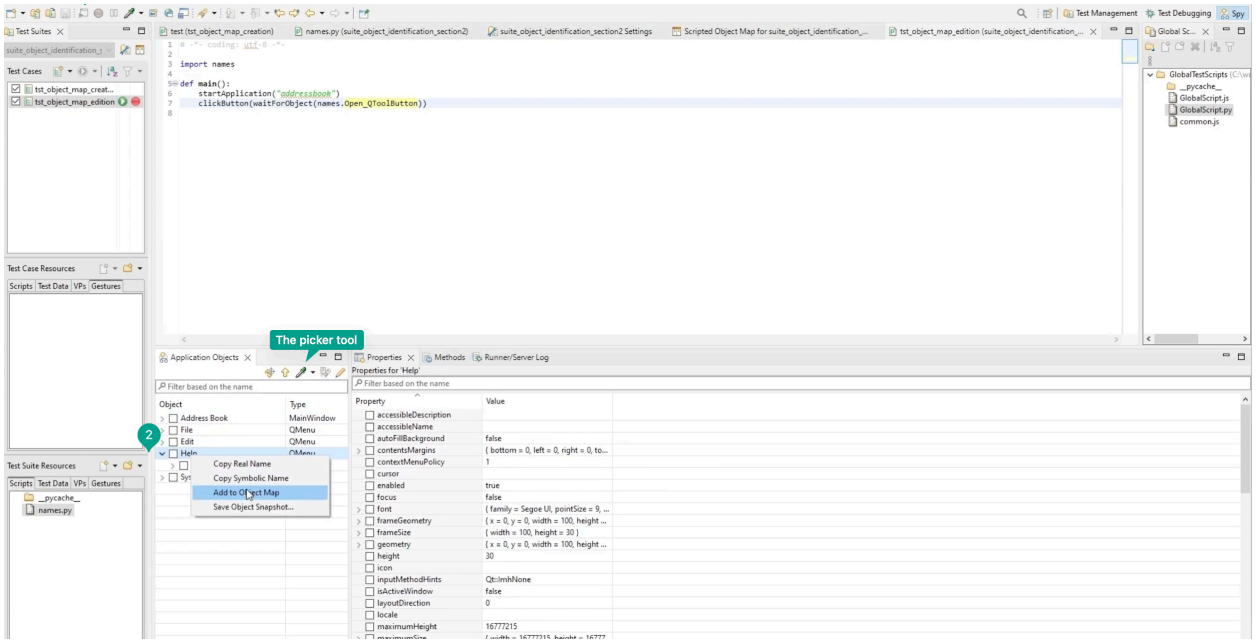
2. Then, we created a new test and in the main function wrote:

startApplication("addressbook")

clickButton(waitForObject(names.Open_QToolButton))

3. We ran the test. Squish clicked on our Open button.

04 Adding new object to the map automatically



1. We started the AUT once more.

2. Then we spied the AUT. We just went through the objects tree in the Application Objects view to find the object we wanted to add or pick with the Picker tool.

3. We selected the Help action from the Address book.

4. In the Application Objects window, we right-clicked on the object and selected Add to Object Map.

The object was now referenced in the object map, ready to be used in the test scripts.

Interacting with Objects

Obtaining Reference to an Object

Once the Object Map is prepared, its names can be used in the script as parameters for the Squish's object lookup functions. These functions return a reference to an object in the UI that can be interacted with in the test scripts.

The standard lookup functions used by Squish during the recording of a snippet are waitForObject(objectOrName) or waitForObjectItem(objectName, itemText). These functions wait until the object corresponding to objectName is accessible (i.e., it exists and is visible and enabled) and returns a reference to it.

For objects that are not visible, waitForObjectExists(objectName) can be used instead.

The existence of an object can also be checked with Object.exists(objectName).

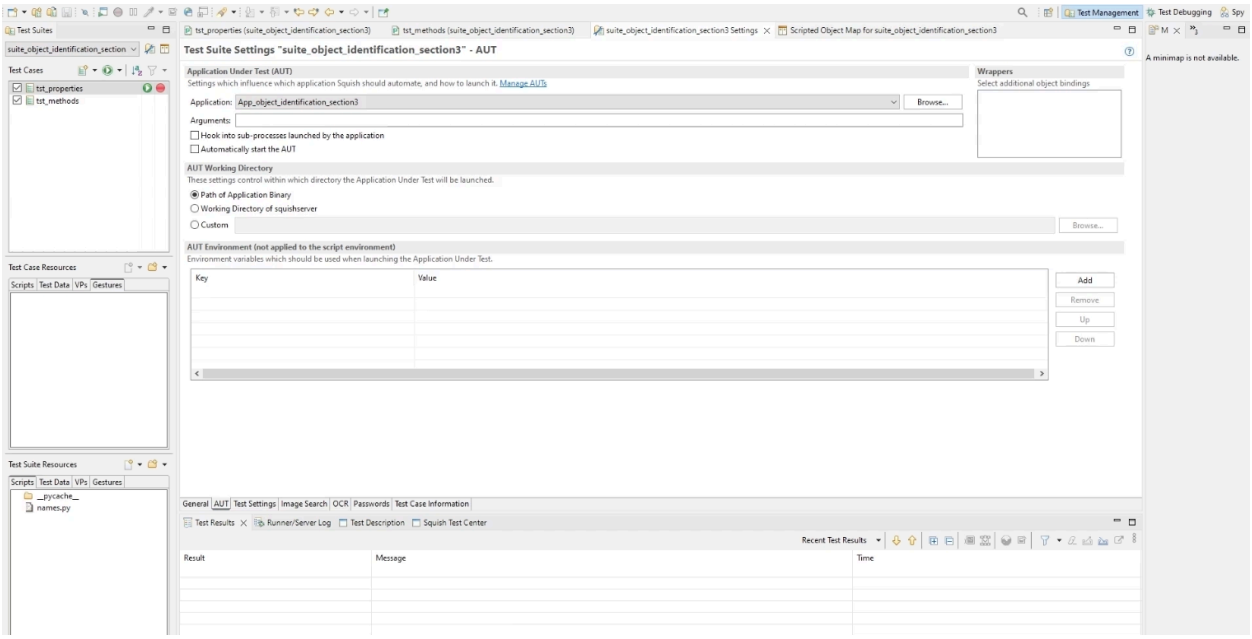
ℹ️ **Once a reference to an object has been obtained with one of the lookup functions, it is possible to interact with the object.**

Object properties can be accessed to get or set their value. Squish recognizes all standard properties of the objects of the Qt Framework. Custom object properties can also be exposed to Squish by declaring them with the Q_PROPERTY macro.

Similarly, **object references can be used to call methods**. Squish recognizes all standard methods of the Qt API. Custom methods can also be called by Squish by declaring them as Qt slots or with the Q_INVOKABLE macro.

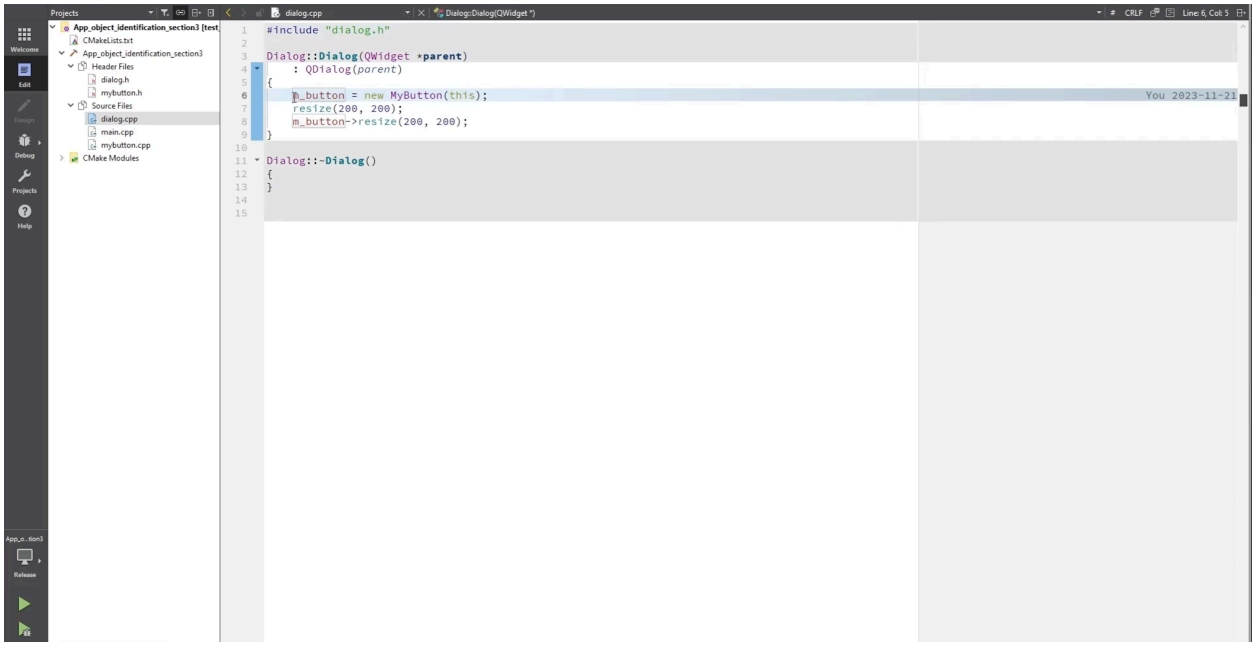
STEPS - How to Interact With Objects That Are Defined Via the Object Map

Starting point



In this example, we used the App_object_identification_section3 application. This application was created specifically for this demonstration. It must be built for the specific Squish version and mapped in the Squish IDE to run the test scripts successfully. The application was built using Qt 6.5.3 MinGw 64-bit.

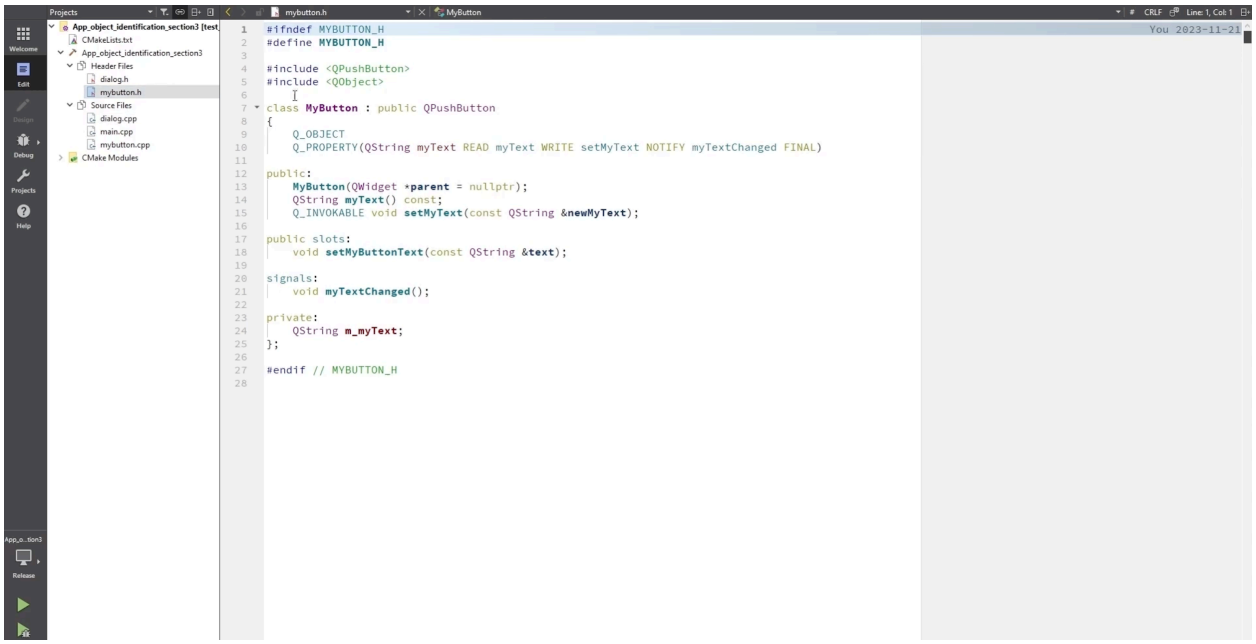
Exploring the example in Qt Creator, part 1



In Qt Creator, we opened the main.cpp, and saw that our UI consisted of a single Dialog instance.

This Dialog contained a single widget, a MyButton object, as could be seen in the constructor of Dialog in dialog.cpp.

Exploring the example in Qt Creator, part 2

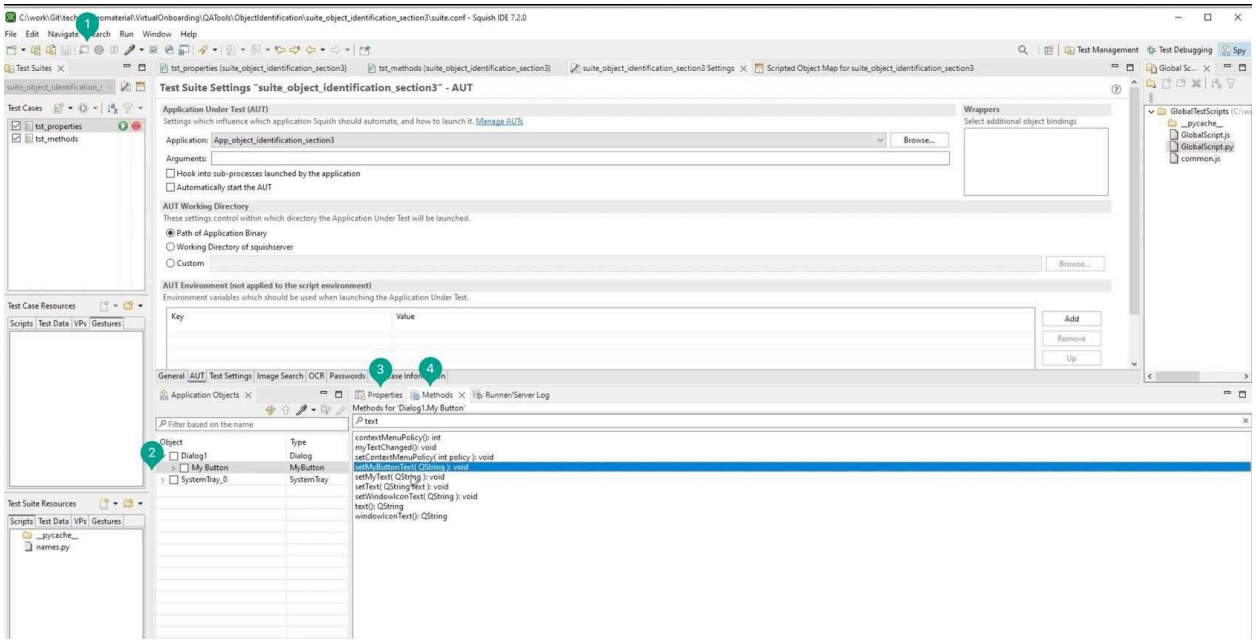


We inspected **mybutton.h**, in Qt Creator. This custom widget inherits from QPushButton. All the properties and methods of QPushButton will therefore be recognized for MyButton by Squish.

We had created a custom property for MyButton, called **myText**, using the Q_PROPERTY macro. This property was defined using the setter **setMyText()** and **getter myText(). All properties defined using Q_PROPERTY will be recognized by Squish.**

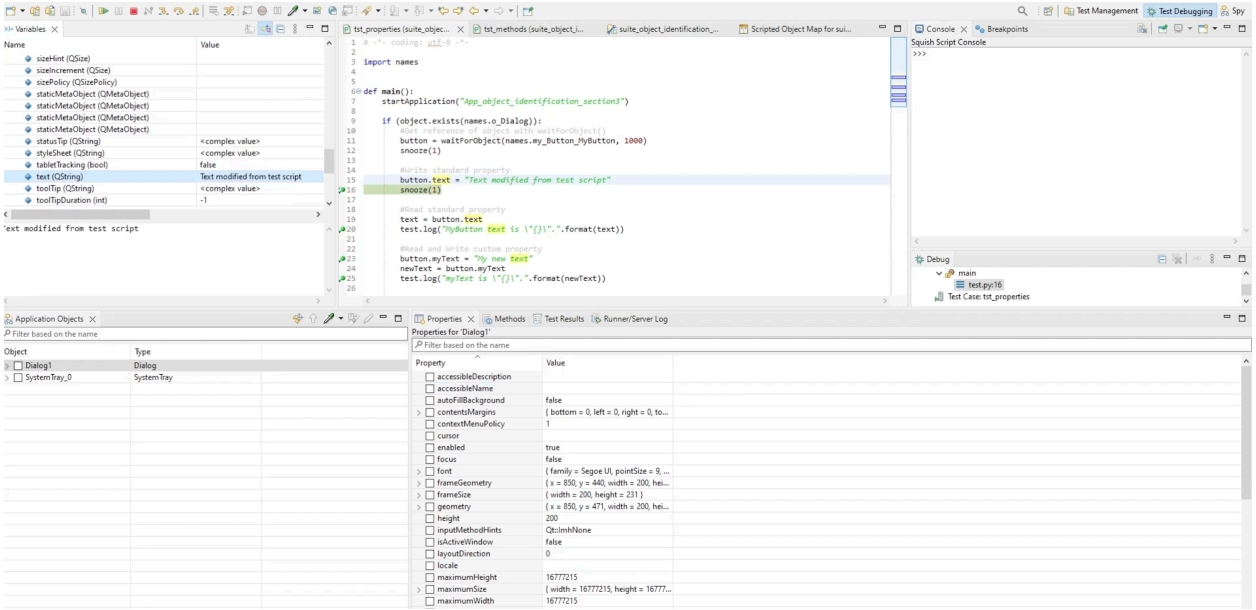
In MyButton, we also had a signal, a slot, and a Q_INVOKABLE method, which would be automatically recognized by Squish.

Exploring the application in Squish IDE



1. We ran the AUT from the Squish IDE.
2. Then, we navigated through the Applications Objects of this AUT. We saw that the MyButton was recognized by Squish.
3. In the Properties window, we checked that the myText property was also recognized by Squish.
4. In the Methods list, you saw that the signal myTextChanged(), the slot setMyButtonText(), and the method setMyText() were all recognized by Squish as methods.

Using the object, properties and methods in the test scripts



Next, we looked at the **tst_properties test**.

1. We started the AUT and verified that the dialog existed. If it did, we would get a reference to the MyButton object called button.
2. Since MyButton inherited from the standard QPushButton, we could use the properties of QPushButton in the script. We modified the text property of the button and set the text variable equal to it:

    button.text = "Text modified from test script"

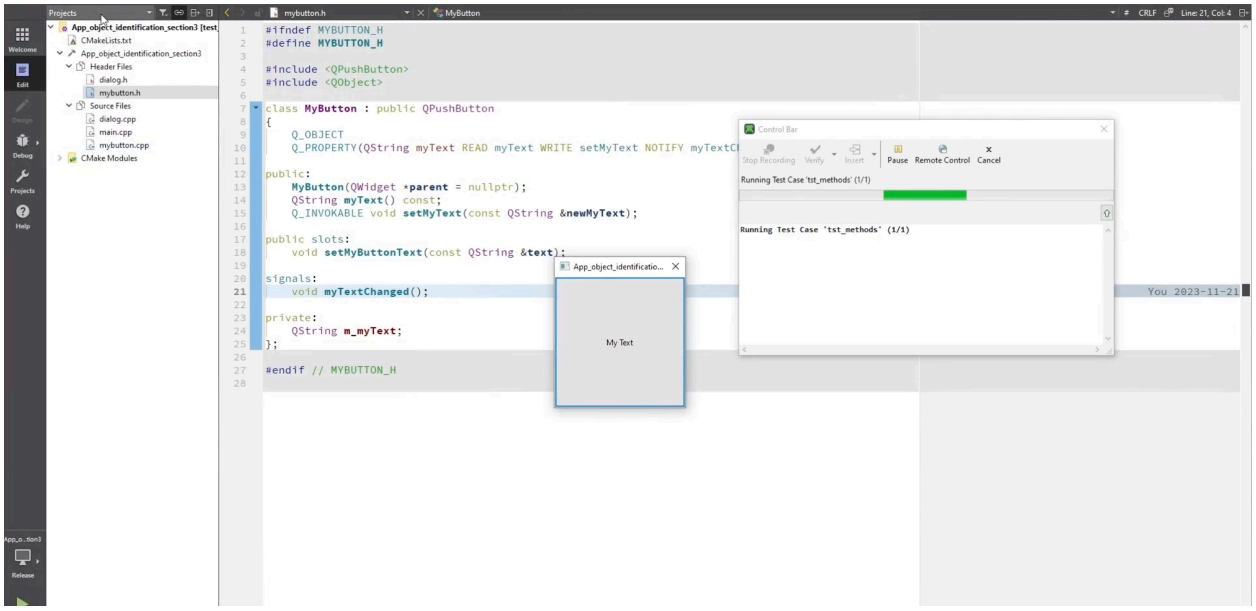
    text = button.text

Since we had used Q_PROPERTY with myText, we could read and write this property with a similar syntax:

button.myText = "My new text"

newText = button.myText

Exploring the tst_methods test



Then, we looked at the **test tst_methods** where we used methods on MyButton.

We started the AUT again. We got the reference to button.

Now, we could call standard methods like in the line **button.setText("Text set from test script")**. We could also call a slot like in the line **button.setMyButtonText(button.myText)**.

Running this test, we saw the methods were correctly called from our test script without any issues.

Dynamic Properties

When interacting with an AUT, it is possible that some object properties used in the real names change in value. This is the case when testing an AUT whose window title changes during the test. This is not ideal because Squish will create several entries in the Object Map for the same object during recording.

This is a relevant example, because as a container, the window can be referenced in real names of other objects so the tester will end up managing multiple references to same objects in the Object Map. Each of these references only work when the AUT is in a specific state.

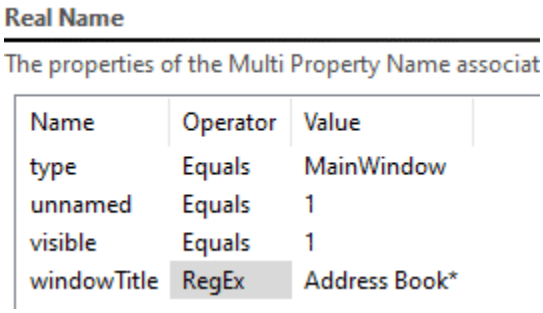
For instance, recording the following test with the Address Book AUT: starting the AUT, clicking and Open to open the address book MyAddresses.adr, and clicking on Open again, two references to the same window and two references to the same button are created as shown:

```python
address_Book_MainWindow = {"type": "MainWindow", "unnamed": 1,
"visible": 1, "windowTitle": "Address Book"}

address_Book_MyAddresses_adr_MainWindow = {"type":
"MainWindow","unnamed": 1,"visible": 1, "windowTitle": "Address
Book - MyAddresses.adr"}
```

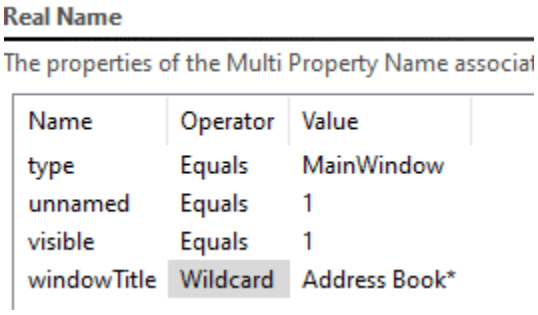
Squish has some solutions to handle dynamic properties and solve the issue:

1. Regular Expressions
    - Can be used in the Object Map by replacing Equals with the RegEx for the Operator
    - The value is the regular expression to use



2. Wildcards
    - Can be used in the Object Map by replacing Equals with the Wildcard for the Operator

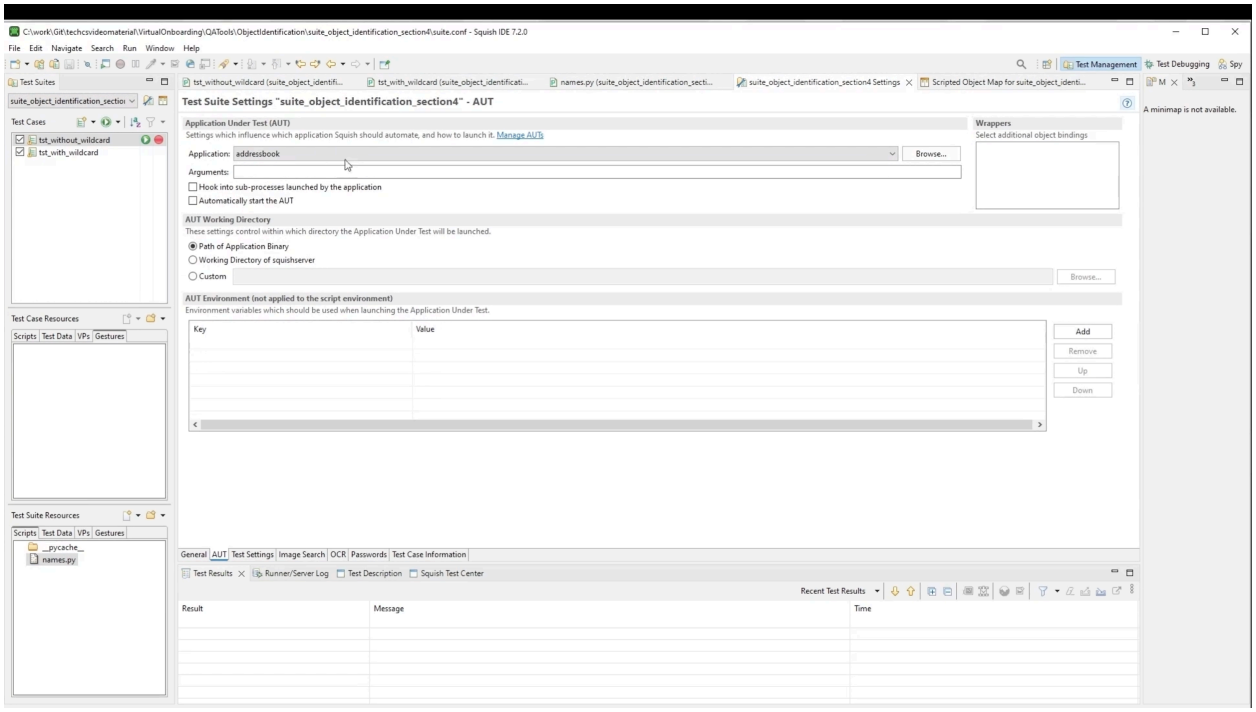
- The value is the wildcard to use



After recording a snippet, it is important to analyze the updated Object Map. There should only be at most one entry per object in the UI, and Squish should be able to access the object regardless of the state of the AUT. If that is not the case, the Object Map should be cleaned up. It is recommended to adjust object names for objects with changeable text at an early stage of the testing process.

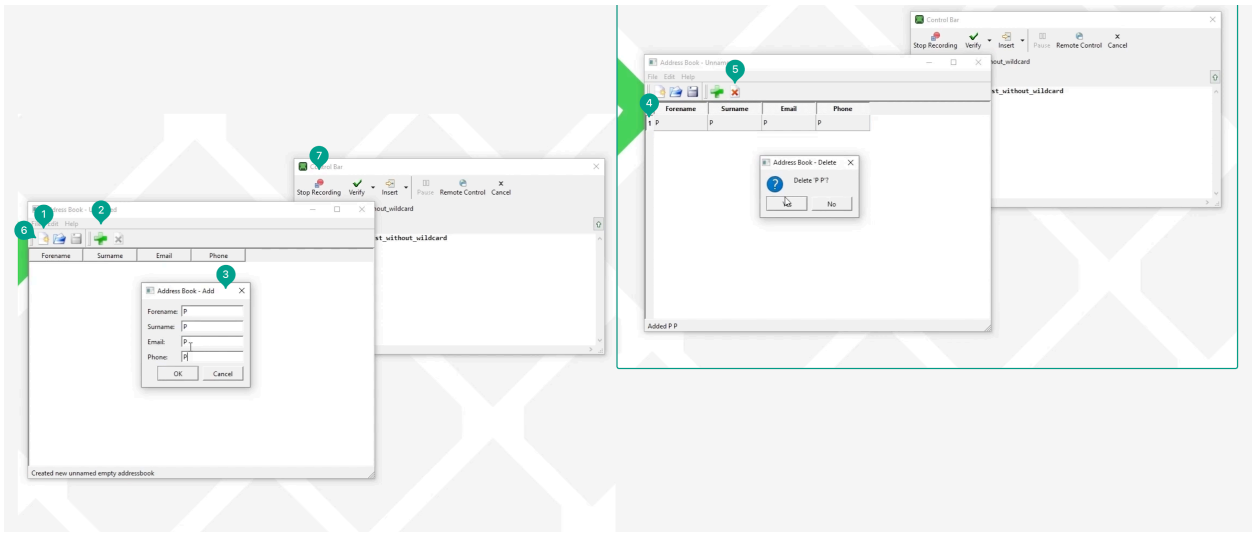
STEPS - How to Handle Dynamic Properties in the Object Map

Starting point



In this demo, we used the Address Book application available in the examples folder of the Squish directory. Here, we had a test suite already opened for this.

Recording a first test to generate the object map



We launched the AUT.

1. We clicked on the New button to create an address book. We noticed that the window title had changed.

2. Then, we clicked the Add button to add a new entry.

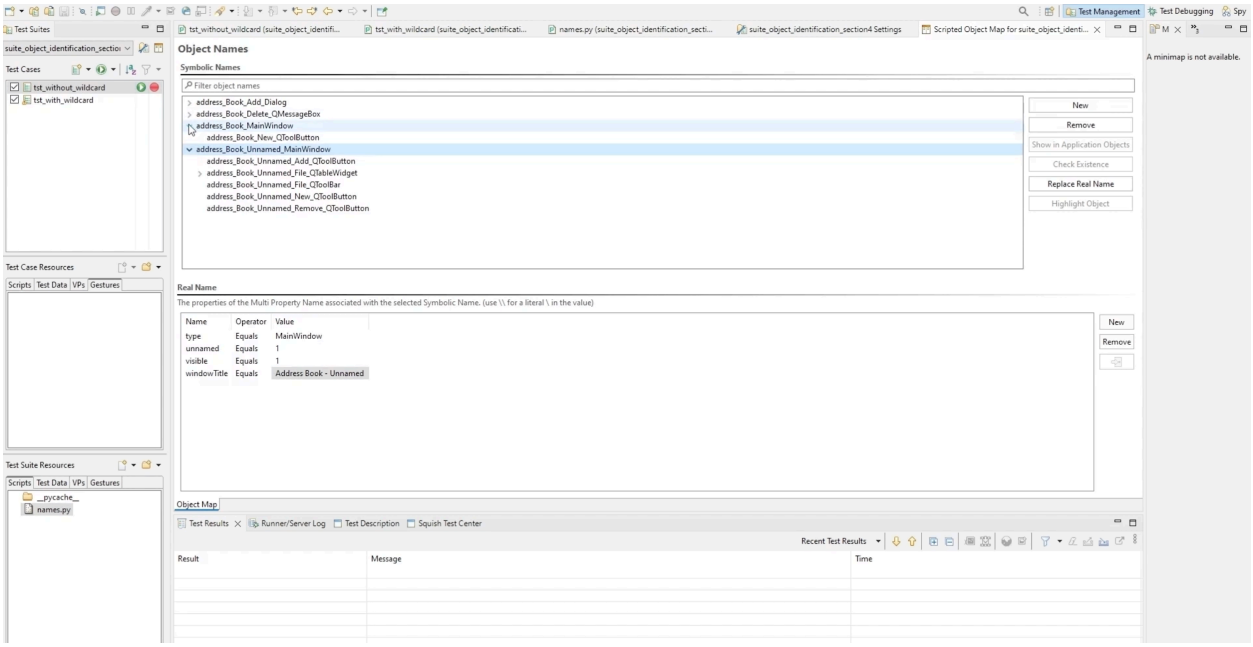
3. We typed the contact information for this entry.

4-5. Next, we selected the added entry and removed it.

6. We clicked on New again.

7. Finally, we stopped the recording

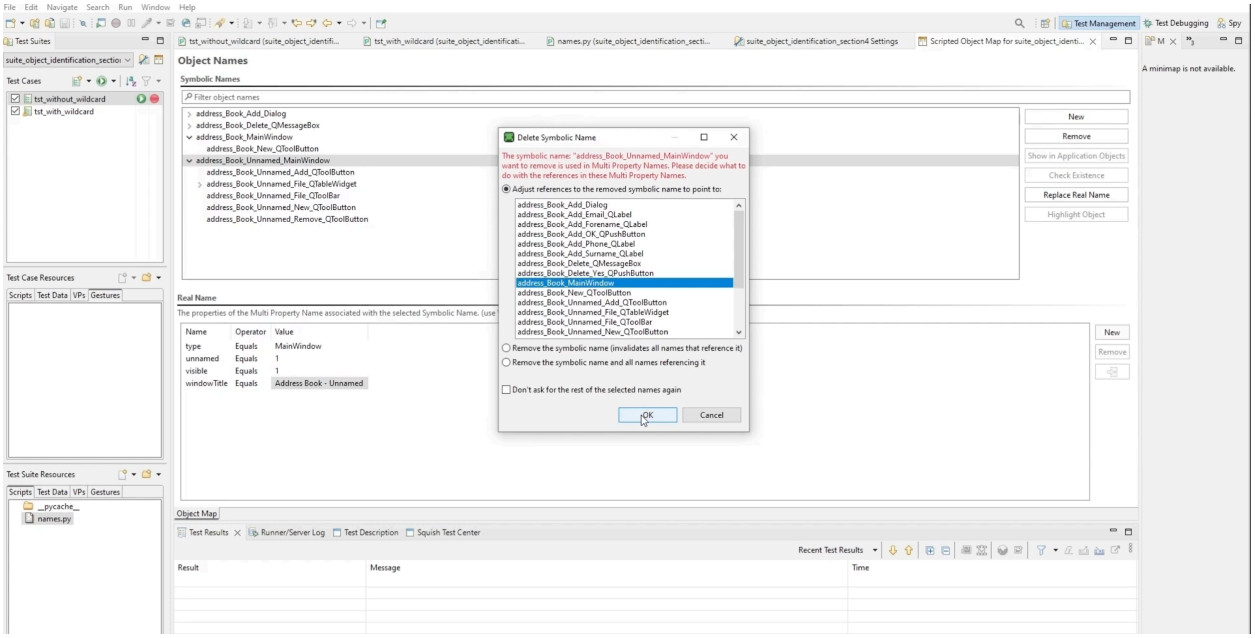
Exploring the object map Squish has generated



We noticed that there were two references for the main window. This was because the windowTitle property was used in the real name of the MainWindow objects. This property was equal to the title of the window. Since the main window was a container for other objects you saw there were also duplicates for the New button.

If you know that an object is the same no matter what the application's state is, you can simplify things and use a single name rather than lots of different ones. Using a single entry for the same UI object can help make test scripts more robust in the face of changes, so it is well worth doing for names which depend on changeable window titles or other volatile text.

Cleaning the object map



The number of entries in the object map does not affect performance of replaying test scripts but affect the recording. The more object names there are to check, the slower the check will be. Duplicated entries can also make the object map unnecessarily large and harder to read.

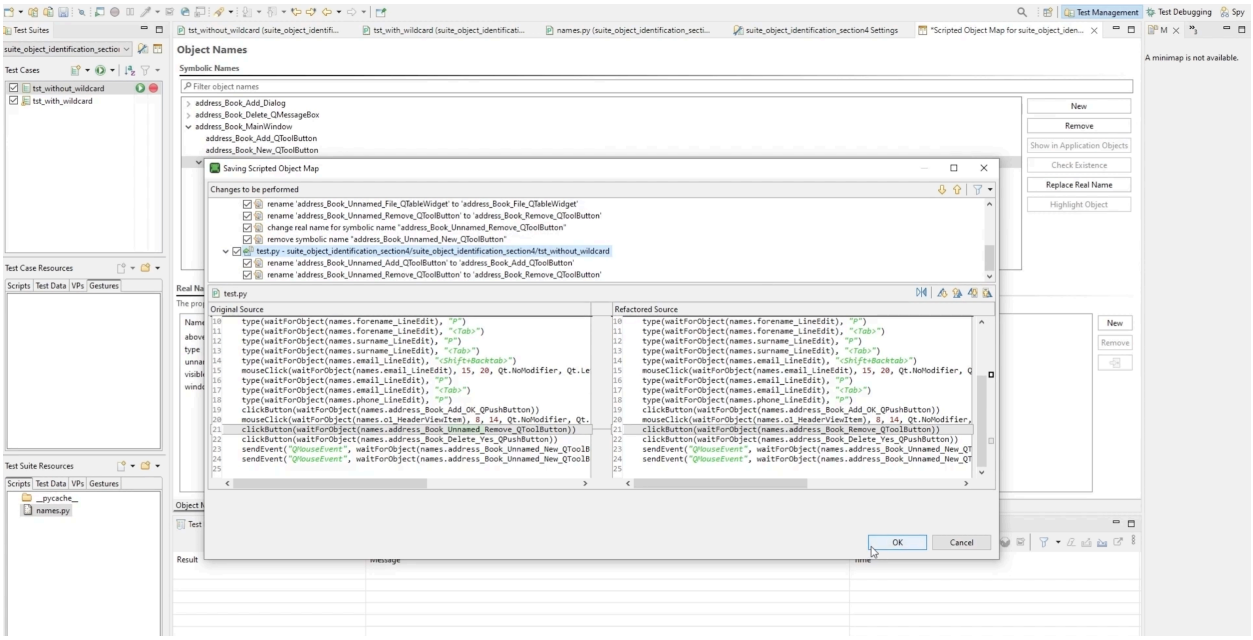
1. We removed the duplicate address_Book_Unnamed_MainWindow.

2. Then, we renamed the objects. This could be done by selecting the object name and doing the wanted chances. In this example, we simply removed the "Unnamed_" part of the name.

3. We deleted the duplicate: address_Book_Unnamed_New_QToolButton.

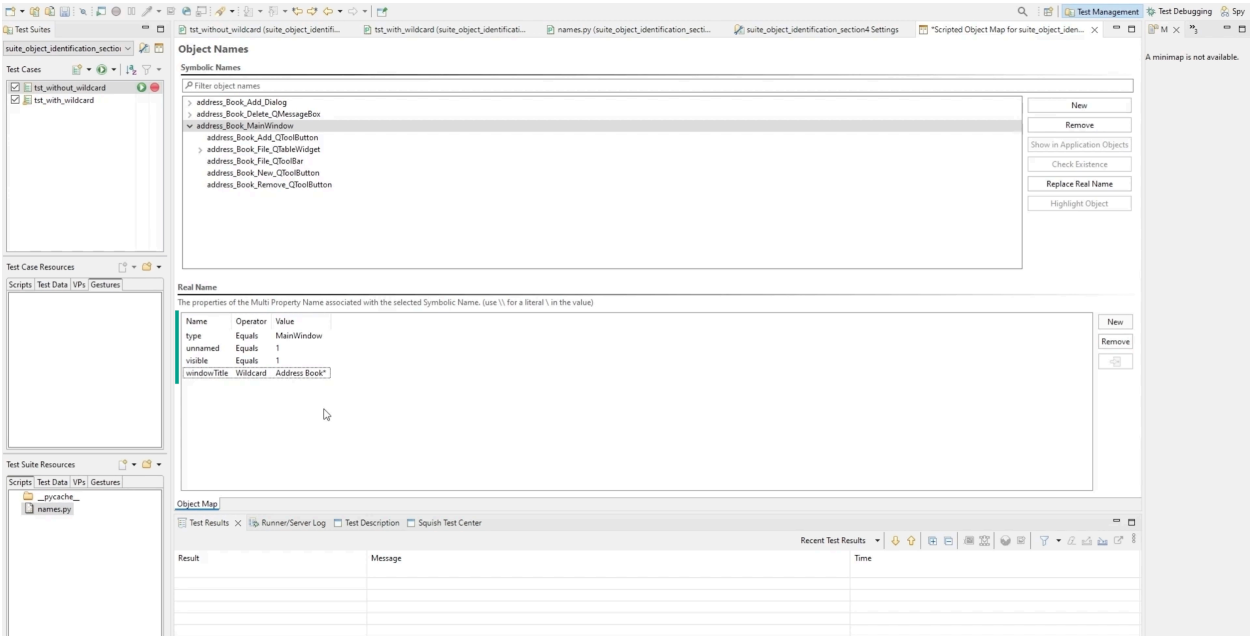
4. Finally, we saved the object map.

Exploring the scripted object map



Squish does some automatic modifications in the scripts. We have now removed the duplicates. However, at this stage, Squish would not be able to find the objects after a new address book has been opened. This is because we were stating in the object map that Squish should look for a window with a windowTitle whose value is Address Book.

Using the Wildcard operator for the windowTitle



We selected the main window in the Object Map view. We saw that there were several options for the Operator.

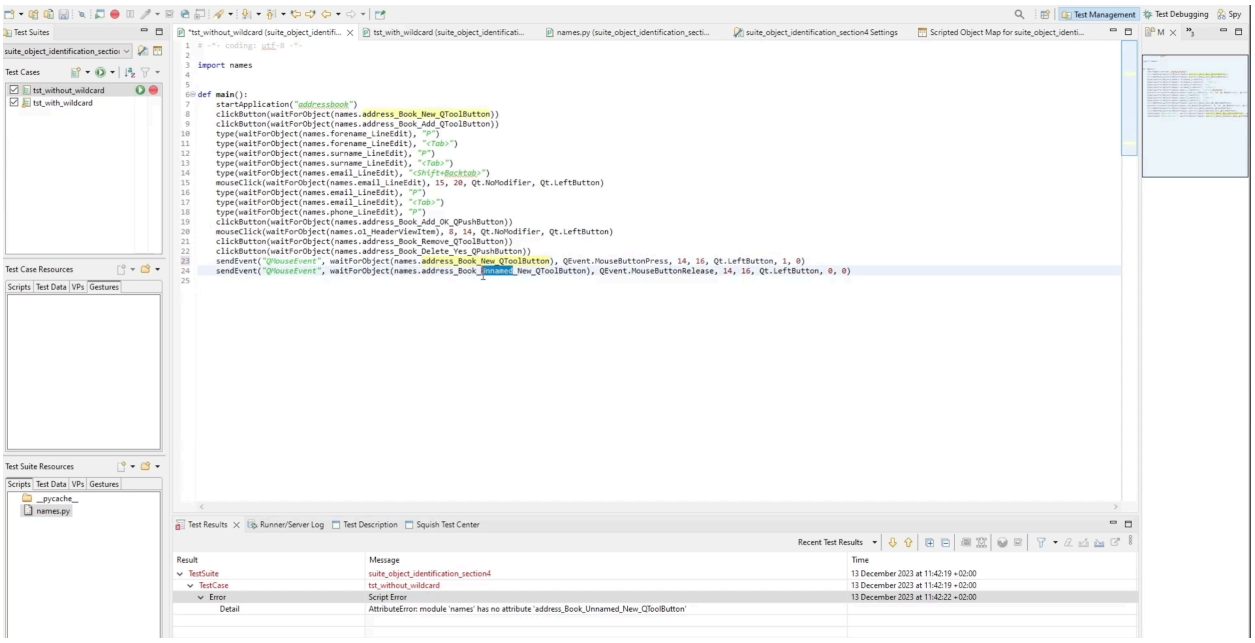
It was set to Equals for the property **windowTitle** but we could use Wildcard or Regular Expression (RegEx).

1. We **used the Wildcard operator and set the value to Address Book***. Then Squish would look for a window whose title starts with Address Book and could be followed by any other text.

2. Then, we saved the object map.

If we opened **names.py**, you saw that the Wildcard() function was being used in the real name of address_Book_MainWindow.

Removing the references to the duplicate of New button in the scripts



Sum up

We had now only one reference per object in the object map and we were using a wildcard to handle the dynamic property used in the real name of the MainWindow. The object map was properly set, and we were therefore ready to run the test.

Object Name Generation

As we have seen in previous sections, **Squish uses a selection of properties to identify objects in an AUT**. Sometimes the properties chosen by Squish by default are not optimal.

For this reason, **Squish allows the user to select which properties should be used to identify objects using a descriptor file (.xml).** This file defines the properties to be used when generating real names for a given type of object. One thing to bear in mind is that Squish takes the inheritance into account. Therefore, when adding a property as an identifier for a type, the property will also be used as an identifier for the classes inheriting that type. It is still possible to exclude some properties if a property needs to be used for an ancestor type but not for a type that inherits it.

It is also possible to apply constraints such that only objects of the given type that meet the constraints are identified.

ℹ️ The default location for the descriptor file is:

SQUISHDIR/etc/qtwrapper_descriptor.xml

SQUISHDIR stands for the directory where the Squish package is installed. It is possible to modify this file directly.

ℹ️ User-specific descriptors file can also be created in

<Squish_User_Settings>/qtwrapper_user_descriptors.xml

<Squish_User_Settings> stands for the directory where user specific settings are stored. On Windows, this is %APPDATA%\froglogic\Squish, and on Unix-like systems, such as Linux and macOS, it is ~/.squish. The environment variable SQUISH_USER_SETTINGS_DIR can also be set and used instead.

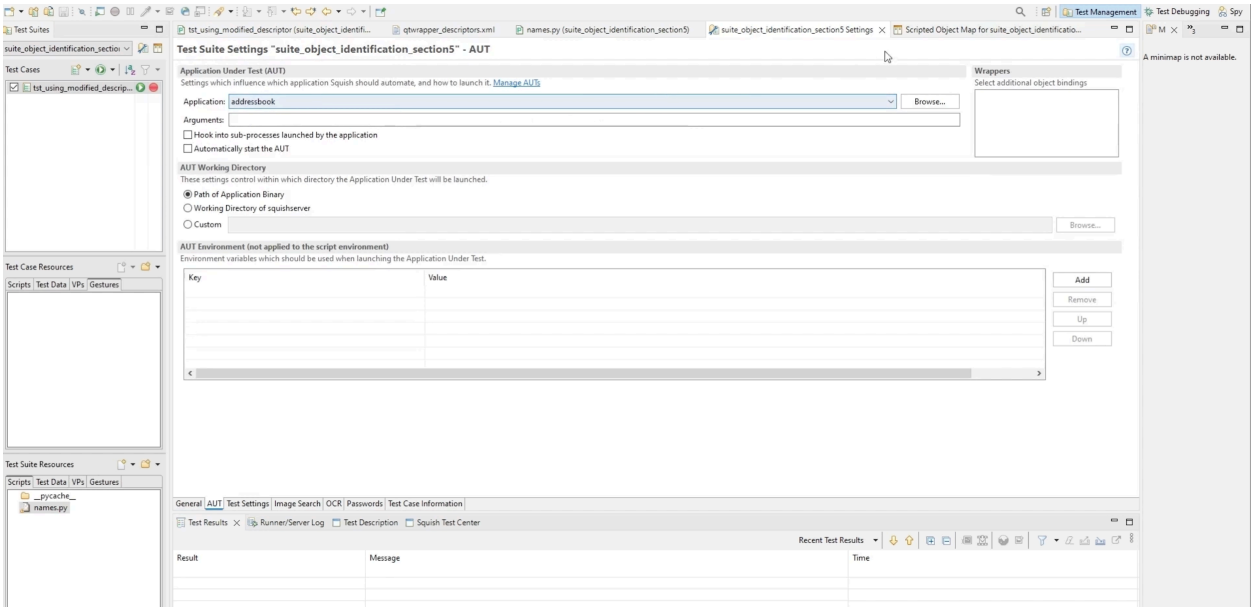
For instance, the tester could create a specific descriptor file which excludes the use of windowTitle for MainWindow types:

```
None
<descriptor>
    <type name="MainWindow"/>
    <realidentifiers>
            <property exclude="yes">windowTitle</property>
```

```
    </realidentifiers>
</descriptor>
```

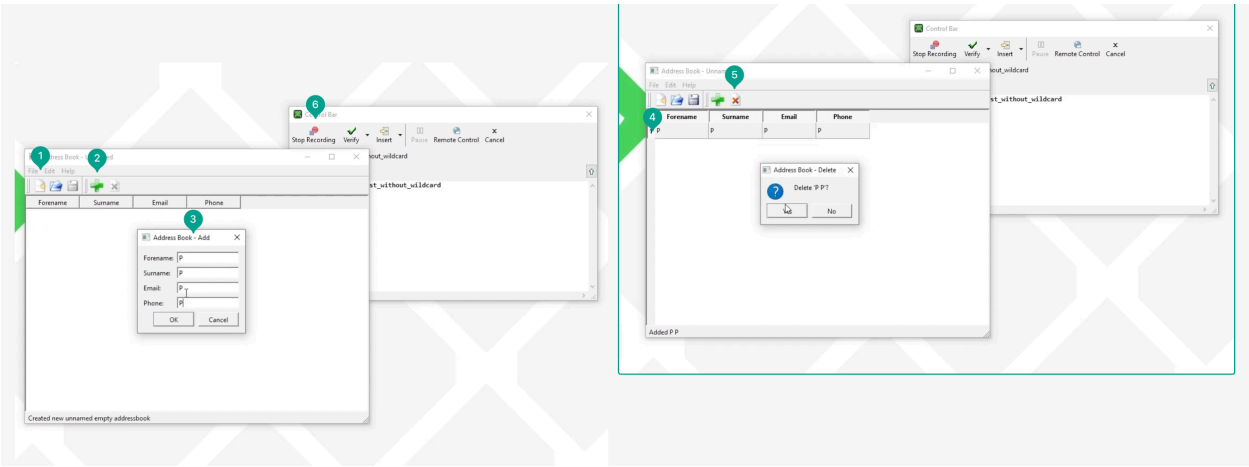
STEPS - How to Modify the Descriptor File

Starting point



In this demo, we used the Address Book application available in the examples folder of the Squish directory. We already had a test suite set up, so we started by recording a test to generate our object map.

Recording a test



We launched the AUT.

1. Then, we clicked on the New button to create an address book. You could notice that the window title had changed.

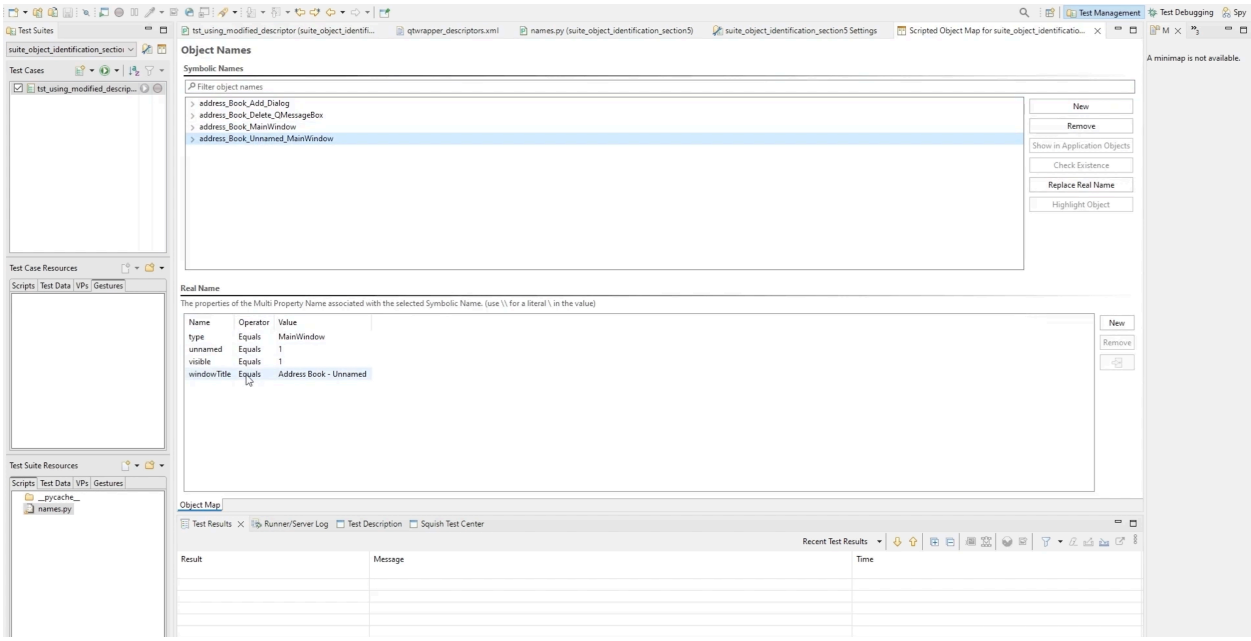
2. We clicked the Add button to add a new entry.

3. Next, we typed the contact information for this entry.

4-5. We selected the added entry and removed it.

6. Finally, we stopped the recording

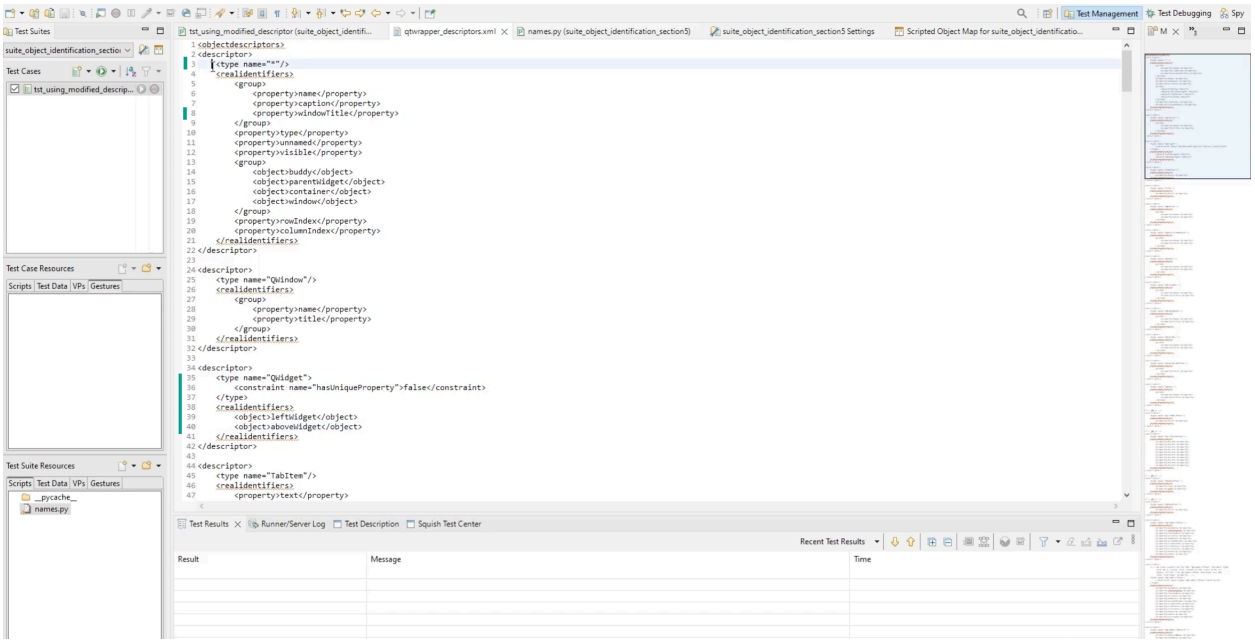
Exploring the object map



Next, we explored the object map Squish had generated. We saw that for the main window object, Squish was using in the real name the properties: **type**, **unnamed**, **visible** and **windowTitle**. We would have liked that the windowTitle was, by default, not used as a property in the real name.

The properties chosen to be part of the real name were defined in what was called a descriptor file.

Exploring the descriptor file



We opened the **SQUISHDIR/etc/qtwrapper_descriptor.xml** file.

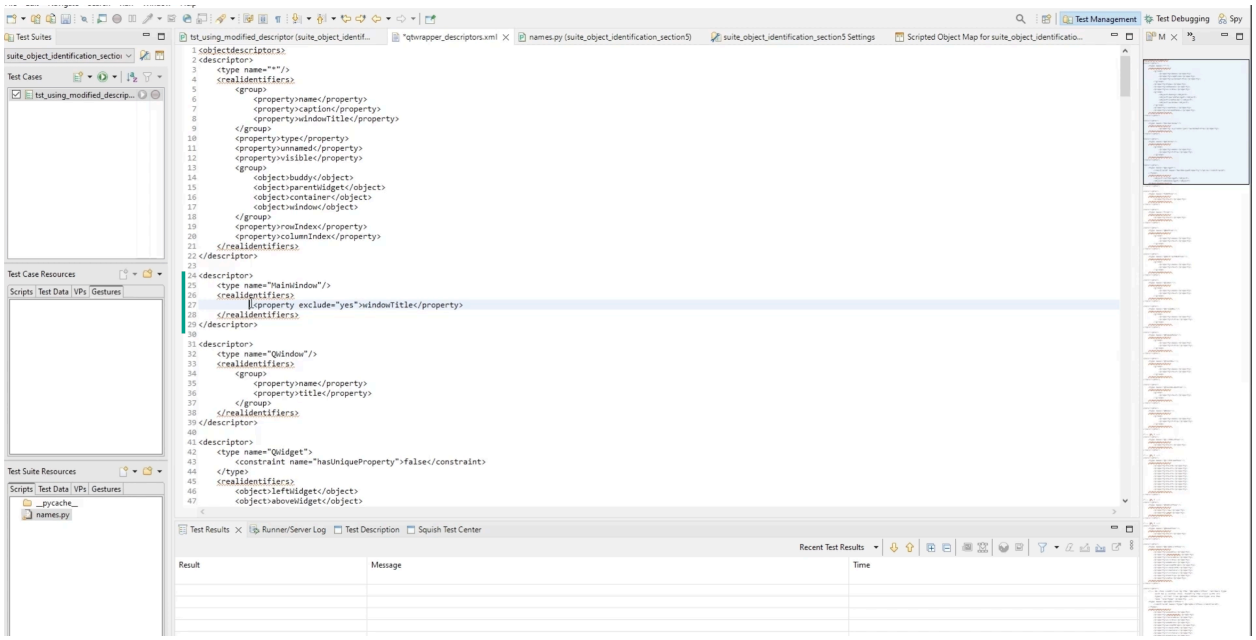
We saw that for most of the usual types, there was a corresponding <descriptor> entry where the properties to use when generating a real name for an object of this type are set.

For instance, for a **QWidget**, or any of the class inheriting Squish used the **leftWidget** and **aboveWidget** in the real name when hasUniqueProperty was false.

There was also the **catch-all * descriptor**. This was where we could define the properties to use for any type. If a type had a property listed here, the property would be used in the real name for the type.

We saw that we had the **windowTitle** listed here. We could have removed the windowTitle from here, but that would have meant that you could not use it in any other type either.

Adding a new descriptor for MainWindow



Then, we added a new descriptor for MainWindow. The idea was to exclude the use of the windowTitle in the names of MainWindow objects. To do this, we needed to modify the descriptor.

To inform Squish that the descriptor should be used for MainWindow objects, we wrote

<type name="MainWindow"/>

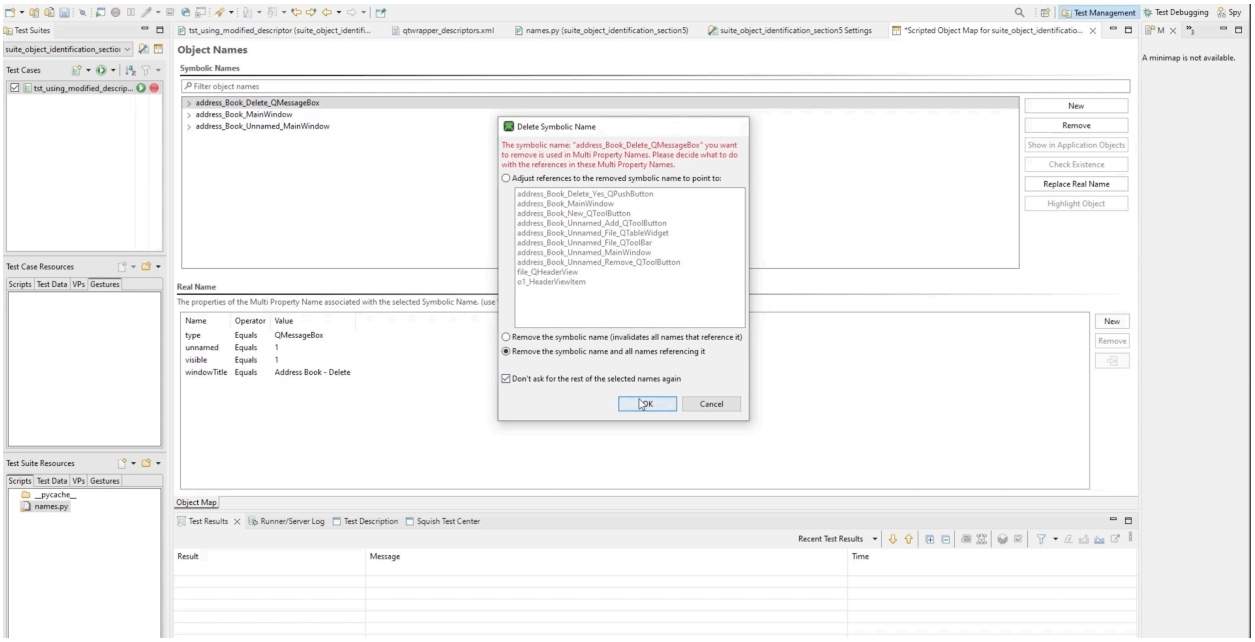
To tell Squish not to use the windowTitle property, we added

<property exclude="yes">windowTitle</property>

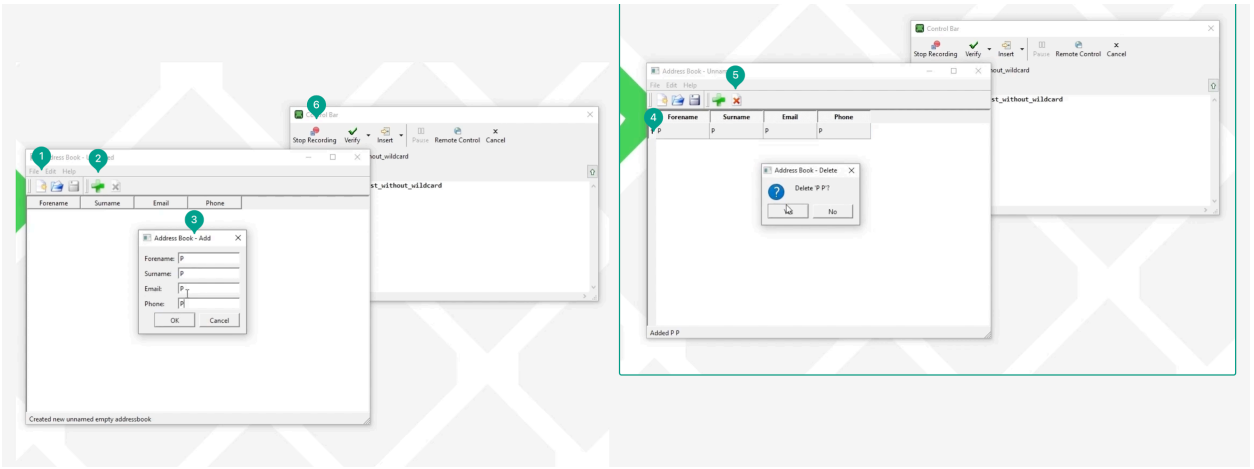
This way, we created a specific descriptor for MainWindow.

Then, we saved the descriptor file.

Cleaning the object map



Recording the same test case as earlier



We started Squish again and went back to the test case.

We launched the AUT.

1. Then, we clicked on the New button to create an address book. We noticed that the window title had changed.

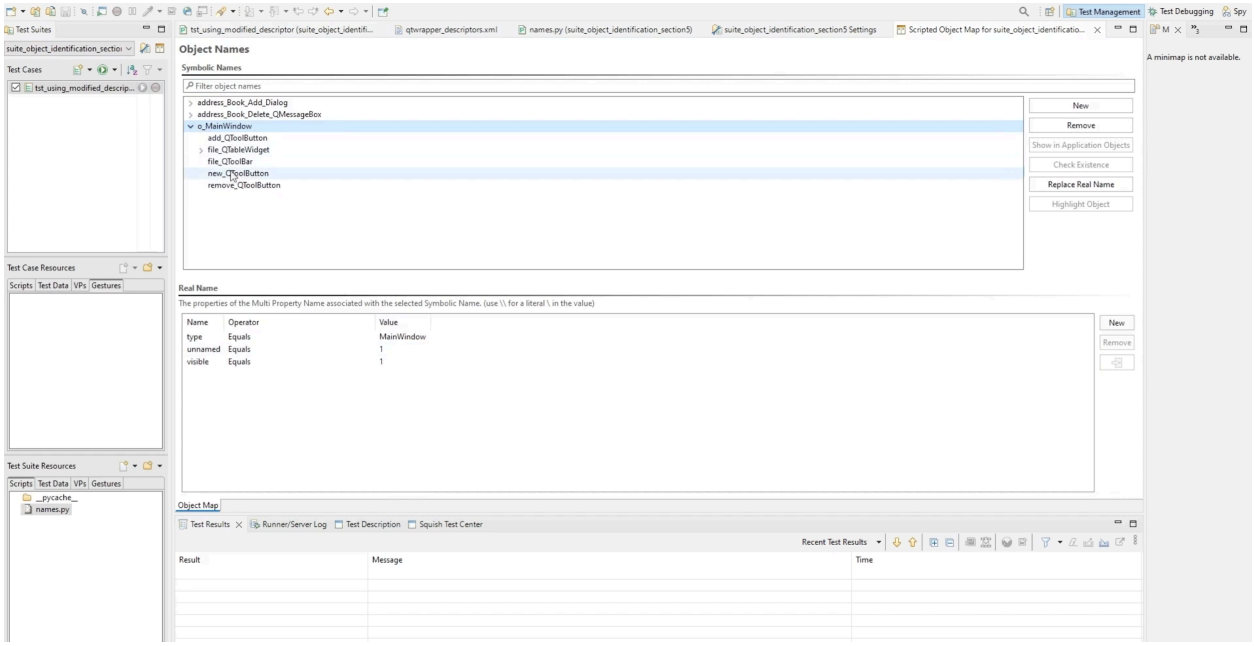
2. We clicked the Add button to add a new entry.

3. Next, we typed the contact information for this entry.

4-5. We selected the added entry and removed it.

6. Finally, we stopped the recording

Exploring the object map



We explored the object map. We saw that the windowTitle was not used anymore in the real name of our main window. This way, we had a cleaner object map without any duplicates.

Be cautious when removing or adding properties to be used in the real names since what works well for one AUT may not for a different AUT.

Occurrences

When an AUT creates two instances of an object whose properties used in the real names have the same values, they will end up with the same property set. In this case, Squish will automatically add an occurrence property to properly identify them. Squish counts the number of times an “object with a set of properties occurs” and assigns it a number.

This is not ideal and can make a test fragile. **When having an occurrence property used in the Object Map, a warning symbol is displayed.** It is suggested that additional properties are used to distinguish objects from each other. A common property that can be used with Qt objects is the object name. It should be used as a unique identifiant for each object to prevent the need for an occurrence property. This is done by the developer in the source code not by the tester.

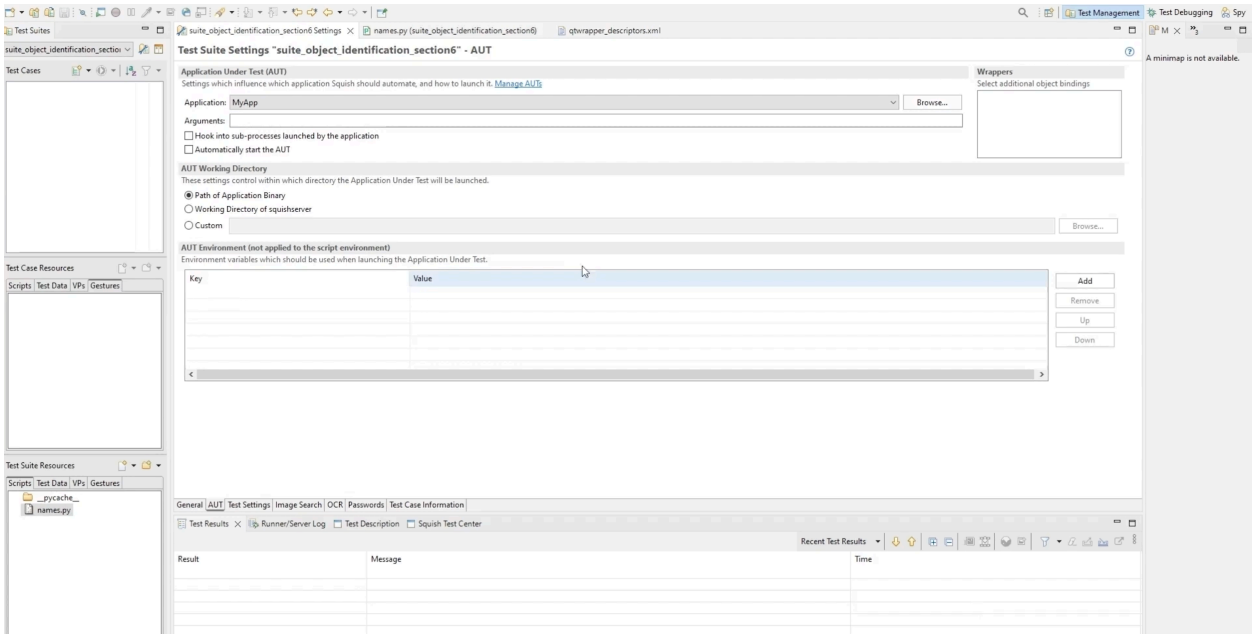
The tester might need to add in descriptor file:

```
None
<property>objectName</property>
```

To make sure the objectName property is used for types with occurrences.

STEPS - How We Can Remove Occurrences From the Object Map

Starting point

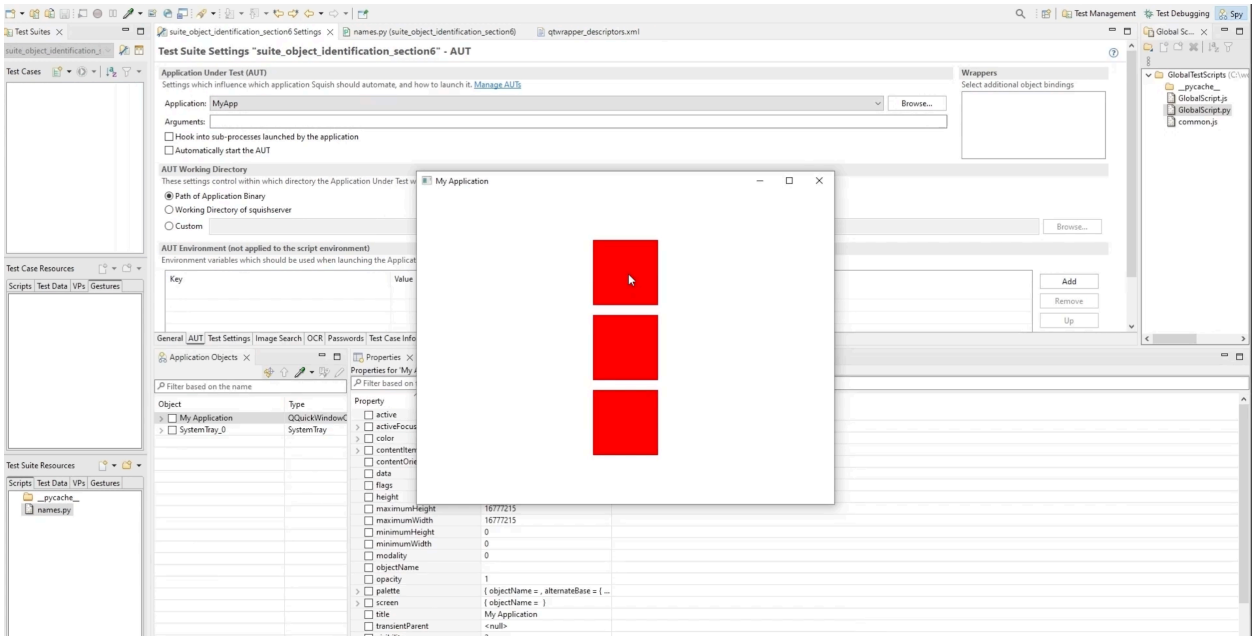


We used the custom Qt Quick application located in App_object_identification_section6.

This application was created for this demonstration and must be built for the specific Squish version and mapped in Squish to run the test scripts successfully. The application was built using Qt 6.5.3 MinGw 64-bit.

First, we looked at how this application was defined in QML and opened the Main.qml. We saw that the UI was just a Window with a Column containing a Repeater to create three Rectangles of the same size and color.

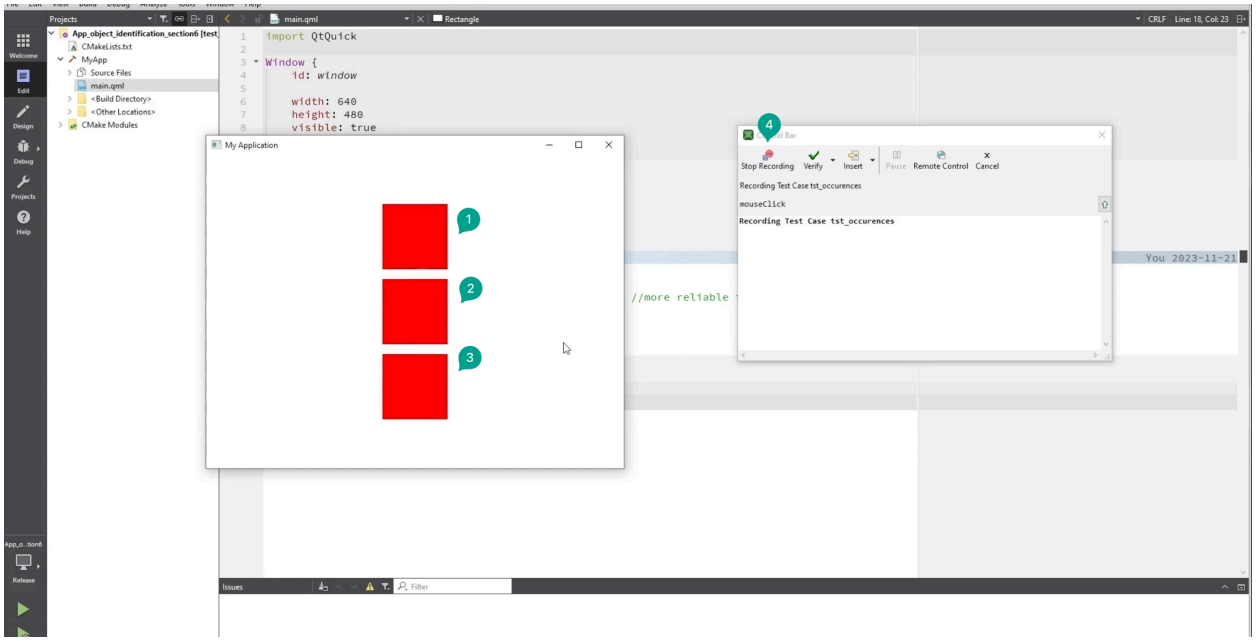
Running the application from Squish IDE



We ran the application from the Squish IDE.

We saw that we had the three red rectangles ordered vertically.

Creating a simple “tst_occurences” test script.



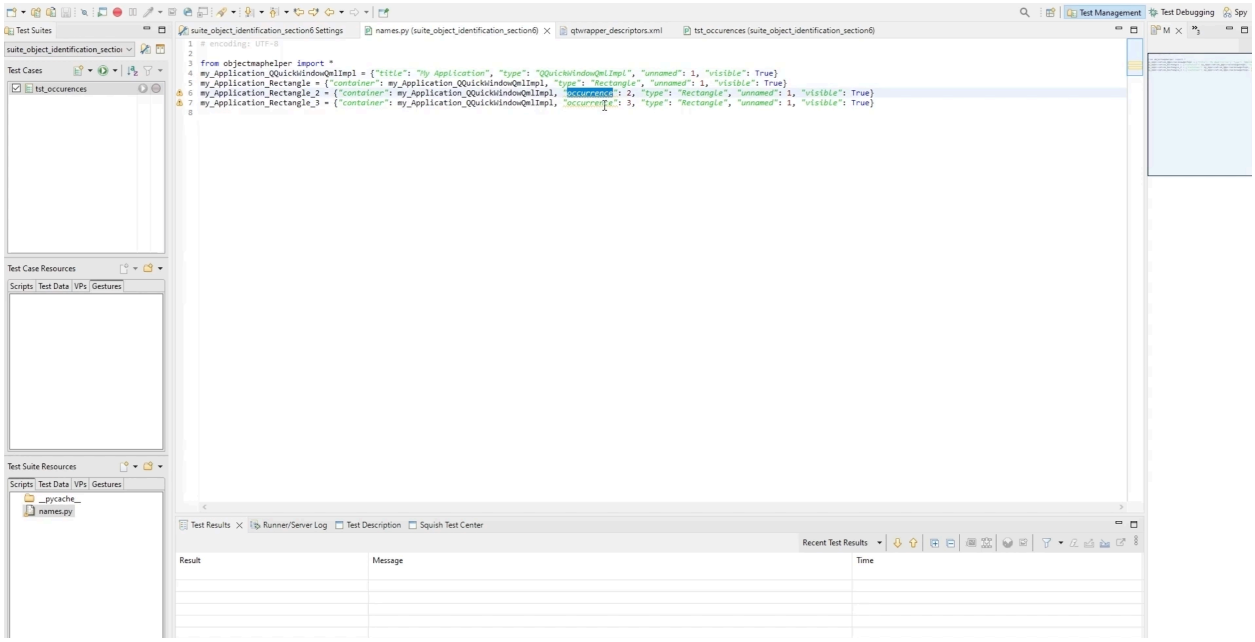
Next, we created a simple “tst_occurences” test script.

1-3. We started the recording and clicked on the three rectangles, one after the other.

4. Then, we stopped the recording.

When the recording stopped, the test script had been successfully generated, but we had a warning symbol next to the names.py.

Exploring the names.py file

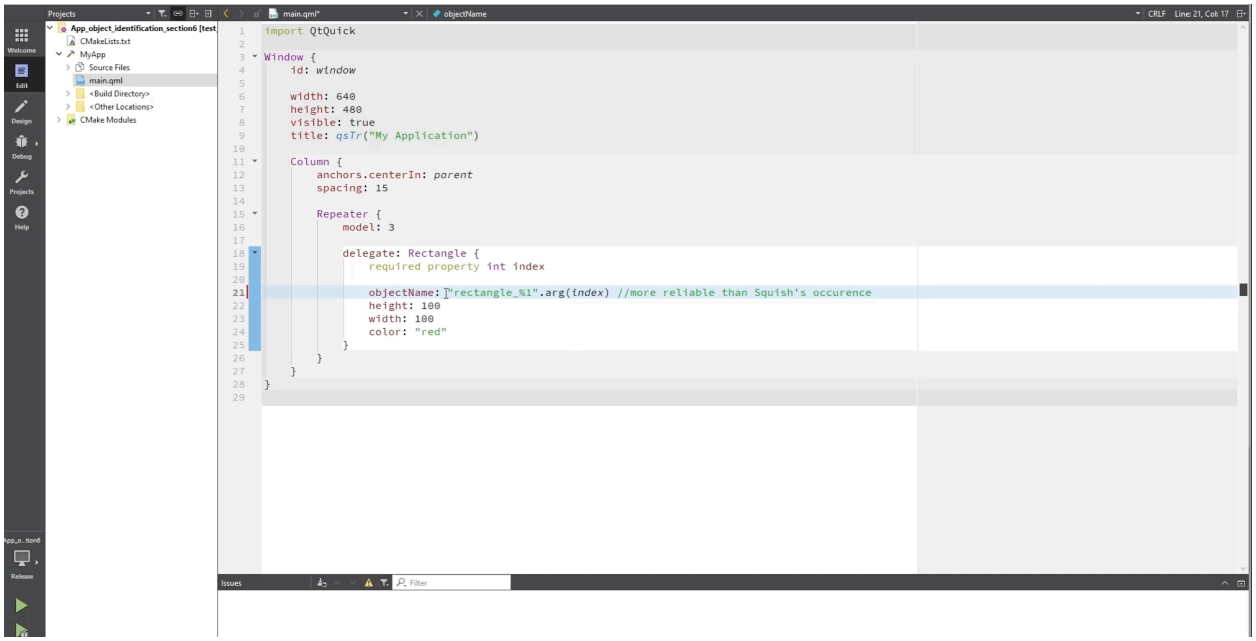


We opened the names.py file.

We saw that Squish had created the object map, but there were some warning signs next to the **my_Application_Rectangle_2** and **my_Application_Rectangle_3**. This was because the real names contained the **occurrence property**. **Squish had added this property to the real name to distinguish each rectangle object between each other**. Indeed, all their properties were the same since they had the same color and were included in the same container.

Occurrence should be avoided: they are added according to the chronological execution of the test script. This makes them less reliable than other properties used to identify objects. For instance, if several test scripts interact with the same objects but in different orders, playback problems can occur.

Identifying the rectangles without using occurrences



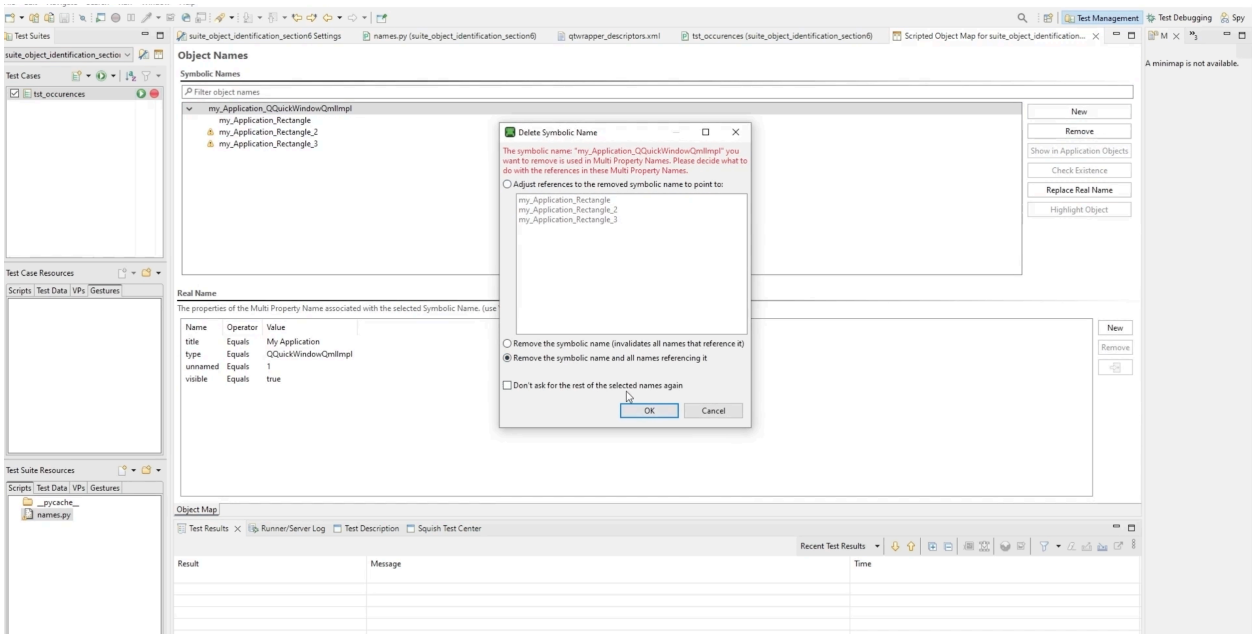
Removing the occurrence property from the real names would not be enough so, we needed **to use other properties to identify the object instead**.

We went back to the **QML file**, **defined the objectName which served as id**. We did not use the QML id here. In this way, we could use the index property in it to distinguish each rectangle in the repeater.

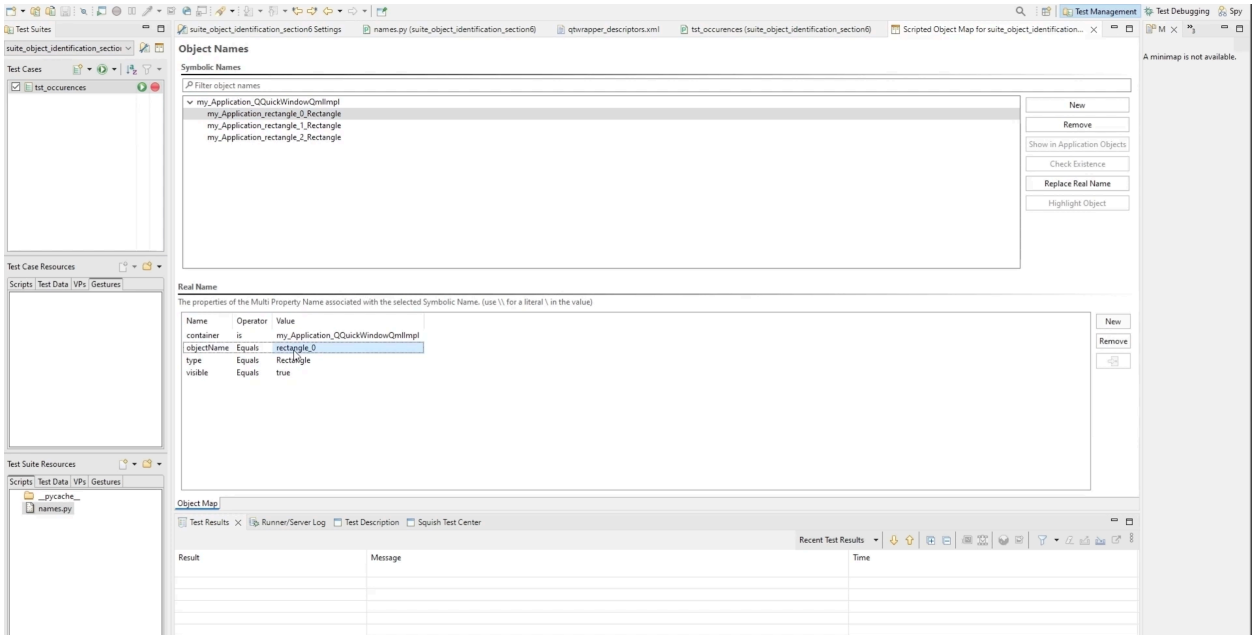
We added **objectName: “rectangle_%1”.arg(index)**

We built the application again.

Clearing the object map



Recording the test again



We recorded the test again by clicking on the three rectangles.

We saw that now the objectName was used instead of the occurrence property, and the tester did not have to add properties manually.

The use of objectName as an identifier is a good practice when testing Qt applications.

Objects can be accessed through the Object Map

Squish relies on the **Object Map** for accessing objects in the user interface. The Object Map is a maintainable repository which defines names for the individual objects in an Application Under Test (AUT).

There are two naming schemes in Object Map

Two naming schemes are used in the Object Map: **Symbolic and Real Names**. Symbolic name refers to a string key which represents an AUT object. Real name, or multi-property name, refers to a set of object properties used to identify an AUT object. Each entry in the Object Map consists of a pair symbolic name-real name.

Once a reference to an object has been obtained with one of the lookup functions, it is possible to interact with the object.

Object properties can be accessed to get or set their value. Squish recognizes all standard properties of the objects of the Qt Framework. Custom object properties can also be exposed to Squish by declaring them with the Q_PROPERTY macro.

Similarly, object references can be used to call methods. Squish recognizes all standard methods of the Qt API. Custom methods can also be called by Squish by declaring them as Qt slots or with the Q_INVOKABLE macro.

Analyzing the updated object map is important

After recording a snippet, it is important to analyze the updated Object Map.

There should only be at most one entry per object in the UI, and Squish should be able to access the object regardless of the state of the AUT. If that is not the case, the Object Map should be cleaned up. It is recommended to adjust object names for objects with changeable text at an early stage of the testing process.

You can select which properties should be used to identify objects using a descriptor file

Squish uses a selection of properties to identify objects in an AUT. In addition, **Squish allows the user to select which properties should be used to identify objects using a descriptor file (.xml).** This file defines the properties to be used when generating real names for a given type of object. One thing to bear in mind is that Squish takes the inheritance into account.

It is also possible to apply constraints such that only objects of the given type that meet the constraints are identified.

Occurrences are used to identify an object which would otherwise have the same real name than another object

When an AUT creates two instances of an object whose properties used in the real names have the same values, they will end up with the same property set. In this case, **Squish will automatically add an occurrence property to identify them**. Squish counts the number of times an "object with a set of properties occurs" and assigns it a number.

Occurrence should be avoided. It is suggested that additional properties are used to distinguish objects from each other. A common property that can be used with Qt objects is the object name. It should be used as a unique id for each object to prevent the need for an occurrence property. This is done by the developer in the source code not by the tester.

The creation and edition of the Object Map can be reduced to the following process:

Record a dummy test Record a dummy test and interact with all the object that will be required in the tests.
Check the Object Map automatically generated by Squish Remove duplicates (e.g.: use RegEx or Wildcards). Remove occurrences (e.g.: add properties in the Object Map view to distinguish the objects, increase the number of properties to be used in the real names by modifying the descriptor file, add object name property for each object in the source of the code AUT).
Maintain the Object Map as the test scripts and the AUT evolve Always check for duplicates and occurrences. Break up names script into several scripts when it is relevant for maintainability. Modify the descriptor file or create user-specific property references. Check the “sanity checks” article.